

Ex.no:2	EDA-DATA IMPORT AND EXPORT

Aim:

To inspect dataset features, detect null values, and export data into CSV, Excel, and SQLite database formats.

Algorithm:

1. Import pandas and sqlite3 libraries.
2. Read diabetes_prediction_dataset.csv using pd.read_csv().
3. Inspect the data structure using .head(), .tail(), .info(), .describe(), and .isnull().sum().
4. Export the DataFrame to .csv and .xlsx formats using .to_csv() and .to_excel().
5. Connect to food_waste.db using sqlite3 and write the data into an SQL table using .to_sql().
6. Close the database connection and end execution.

Program:

```
import os
```

```
import pandas as pd
```

```
import numpy as np
```

```
path = "."
```

```
print("\nDataset path:", path)
```

```
print("\nFiles in dataset:")
```

```
print(os.listdir(path))
```

```
csv_files = [
```

```
    f for f in os.listdir(path)
```

```
    if f.lower().endswith(".csv")
```

```
]
```

```
if not csv_files:
```

```
    raise FileNotFoundError("No CSV file found in the dataset path.")
```

```
csv_file = [
```

```
f for f in csv_files
    if "diabetes" in f.lower()
[[0]

print("\nCSV file:", csv_file)

df = pd.read_csv(
    os.path.join(path, csv_file)
)

print("\nFirst 5 rows:")
display(df.head())

print("\nLast 5 rows:")
display(df.tail())

print("\nDataset information:")
df.info()

print("\nShape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nStatistical description:")
display(df.describe(include="all").T)

print("\nMissing values:")
```

```
display(  
    df.isnull().sum()  
    .sort_values(ascending=False)  
)  
  
print("\nDuplicate rows:")  
print(df.duplicated().sum())  
  
print("\nGender:")  
display(  
    df["gender"].value_counts()  
)  
  
print("\nSmoking history:")  
display(  
    df["smoking_history"].value_counts()  
)  
  
print("\nHypertension status:")  
display(  
    df["hypertension"].value_counts()  
)  
  
print("\nHeart disease status:")  
display(  
    df["heart_disease"].value_counts()  
)  
  
print("\nDiabetes status:")  
display(  
    df["diabetes"].value_counts()  
)
```

```
print("\nGender-wise diabetes cases:")
display(
    df.groupby("gender")["diabetes"]
      .sum()
      .sort_values(ascending=False)
)

print("\nSmoking history-wise diabetes cases:")
display(
    df.groupby("smoking_history")["diabetes"]
      .sum()
      .sort_values(ascending=False)
)

df["Age_Group"] = pd.cut(
    df["age"],
    bins=[0, 29, 49, float("inf")],
    labels=["Young", "Middle Age", "Senior"]
)

print("\nAge group-wise diabetes cases:")
display(
    df.groupby(
        "Age_Group",
        observed=False
    )["diabetes"]
      .sum()
      .sort_values(ascending=False)
)

print("\nAverage blood glucose level by age group:")
```

```
display(  
    df.groupby(  
        "Age_Group",  
        observed=False  
    )["blood_glucose_level"]  
    .mean()  
)  
  
print("\nAverage BMI by diabetes status:")  
display(  
    df.groupby("diabetes")["bmi"]  
    .mean()  
)  
  
print("\nAverage HbA1c level by diabetes status:")  
display(  
    df.groupby("diabetes")["HbA1c_level"]  
    .mean()  
)  
  
print("\nAverage blood glucose level by diabetes status:")  
display(  
    df.groupby("diabetes")["blood_glucose_level"]  
    .mean()  
)  
  
print("\nCorrelation:")  
display(  
    df[  
        [  
            "age",  
            "hypertension",
```

```

        "heart_disease",
        "bmi",
        "HbA1c_level",
        "blood_glucose_level",
        "diabetes"
    ]
].corr()
)

diabetes_percentage = (
    df["diabetes"].sum()
    / len(df)
) * 100

print("\nDiabetes Percentage:")
print(round(diabetes_percentage, 2), "%")

df["Diabetes_Percentage"] = (
    df["diabetes"] * 100
)

print("\nDiabetes Percentage column:")
display(
    df[
        [
            "gender",
            "age",
            "diabetes",
            "Diabetes_Percentage"
        ]
    ].head()
)

```

```

df.to_excel(
    "diabetes_data.xlsx",
    index=False
)

df.to_json(
    "diabetes_data.json",
    orient="records",
    indent=4
)

df.to_csv(
    "diabetes_data.csv",
    index=False
)

print("\nCSV, Excel and JSON files created successfully.")

```

Output:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 100000 non-null  object
1   age                   100000 non-null  float64
2   hypertension           100000 non-null  int64
3   heart_disease          100000 non-null  int64
4   smoking_history        100000 non-null  object
5   bmi                   100000 non-null  float64
6   HbA1c_level            100000 non-null  float64
7   blood_glucose_level    100000 non-null  int64
8   diabetes               100000 non-null  int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB
Out[4]:
Index(['gender', 'age', 'hypertension', 'heart_disease', 'smoking_history',
      'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes'],
      dtype='object')

```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level
0	female	80.0	0	1	never	25.19	6.6	140
1	female	54.0	0	0	no info	27.32	6.6	80
2	male	28.0	0	0	never	27.32	5.7	158
3	female	36.0	0	0	current	23.45	5.0	155
4	male	76.0	1	1	current	20.14	4.8	155

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose_level
count	96146.000000	96146.000000	96146.000000	96146.000000	96146.000000	96146.000000
mean	41.794326	0.077601	0.040803	27.321461	5.532609	138.218231
std	22.462948	0.267544	0.197833	6.767716	1.073232	40.909771
min	0.080000	0.000000	0.000000	10.010000	3.500000	80.000000
25%	24.000000	0.000000	0.000000	23.400000	4.800000	100.000000
50%	43.000000	0.000000	0.000000	27.320000	5.800000	140.000000
75%	59.000000	0.000000	0.000000	29.860000	6.200000	159.000000
max	80.000000	1.000000	1.000000	95.690000	9.000000	300.000000

	gender	smoking_history
count	96146	96146
unique	3	6
top	female	never
freq	56161	34398

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level
0	female	80.0	0	1	never	25.19	6.6	140
1	female	54.0	0	0	no info	27.32	6.6	80
2	male	28.0	0	0	never	27.32	5.7	158
3	female	36.0	0	0	current	23.45	5.0	155
4	male	76.0	1	1	current	20.14	4.8	155
5	female	20.0	0	0	never	27.32	6.6	85
6	female	44.0	0	0	never	19.31	6.5	200
7	female	79.0	0	0	no info	23.86	5.7	85
8	male	42.0	0	0	never	33.64	4.8	145
9	female	32.0	0	0	never	27.32	5.0	100

Result: